

Las personas son un factor clave en los procesos, también necesitamos herramientas que nos ayuden

Proceso definido: asume que podemos repetir el proceso indefinidamente y **tener los mismos resultados**, usan la predictibilidad para la administración y control. Se adapta el proceso a cada situación particular. Intentan ser procesos completos, que describen lo más posible lo que hay que hacer. La experiencia puede servir para otros proyectos distintos. Creen que se puede detallar de antemano que hay que hacer para realizar el producto.

Procesos empíricos: cubren algunos aspectos, cuando se repite un proceso puede llegar a obtener resultados diferentes. La administración y control es a través de inspecciones. Trabajan en ciclos cortos y usan la retroalimentación. La experiencia es aplicable al mismo equipo, usa la experiencia que va ganando en el proyecto, no la de otros proyectos. El comportamiento de un equipo, para un proyecto, para un producto no es extrapolable.

Ciclo de vida vs proceso: para un proyecto se elige un ciclo de vida para usar el proceso

El ciclo de vida del proyecto te dice cómo encarar las actividades del proceso, cuanto hacer, cada cuanto, ejemplo: cascada, iterativo, espiral (para proyectos)

Hay ciclo de vida de productos (más largos, inician con la idea de crear sw y terminan cuando el producto se retira del mercado ejemplo word) y proyectos (requerimientos, análisis, diseño, etc). En un ciclo de vida de un producto puede haber n ciclo de proyectos (cuando lo mejoro al producto es un nuevo proyecto), cada proyecto genera una versión.

Tipos de ciclos de vida:

- El cascada es secuencial.
- el pud es iterativo/incremental.
- el espiral es el recursivo-> das vueltas pero no generas nuevas versiones del producto, se pone en producción al final recién, solo mostras prototipos.

En procesos definidos puedo elegir cualquier tipo de ciclos de vida y combinarse, procesos empíricos usan el iterativo.

Proceso: conjunto de actividades (estructuradas/guidadas por un objetivo) que toman como entrada un requerimiento y obtienen como salida un producto o servicio. Definición teórica de lo que hay que hacer..

El proceso es el nivel más abstracto, define qué se debería hacer. Ese proceso es el que tengo que adaptar en caso de un proceso definido o terminar de armar en caso de uno empírico con la gente involucrada. El proceso se define en términos de roles, para cada proyecto las personas asumen uno o más roles. El proceso es teórico y yo lo instancio en el proyecto para tener un lineamiento de cada persona que rol tiene.

Proyecto: el medio por el cual obtengo un producto de sw, es la unidad organizativa/de gestión, administro los recursos y personas que van a estar trabajando. Para poder existir necesita personas, recursos y una indicación de cómo hacer y que hay que hacer para alcanzar el producto. El proceso da esta definición teórica, el proyecto toma de ahí lo que le hace falta y arma el conjunto de tareas que son necesarias hacer, incorpora al personal que asumen roles definidos en el proceso y arma el alcance del proyecto (trabajo a hacer para cumplir el objetivo).

Ejemplo: el proceso dice que necesito un arquitecto y en el proyecto asigno a x como arquitecto. Una persona puede asumir más de un rol por proyecto y muchas personas pueden asumir el mismo rol. El proceso dice que tomo requerimientos y el proyecto que x a las 10 am toma requerimientos. El producto es el software (conocimiento empaquetado, ejemplo: código, una bd, el diseño de la bd, un caso de uso, manuales de usuario-> todo lo que sale como resultado de un proyecto).

- Cada tarea tiene un entregable para saber si se hizo o no, cada artefacto que sale es sw y eso se llama ítem de configuración (según la gestión de configuración)

El ítem de configuración tiene importancia para el equipo y por eso se debe mantener.

Scrum no es un proceso, es un framework de trabajo.

SCM (software configuration management): se aplica en todo el proyecto, busca mantener la integridad del software. Es una disciplina de soporte. Cada cosa a controlar es el ítem de configuración.

Mantener la integridad es responsabilidad de todo el equipo.

Integridad es que satisfaga las necesidades del usuario, rastreable en su ciclo de vida, cumplir con la expectativa de costos y satisfacer criterios de performance.

Esta disciplina asegura la calidad del sw, ayuda a que el resto de disciplinas a que puedan hacer su trabajo, es como un contenedor.

- Repositorio: donde se mantienen los ítems, es el contenedor de ítems. Debería tener una estructura, permite compartir.
- Línea base
- Ramas
- Configuración del sw:

software: conocimiento representado en distintos niveles de abstracción y su documentación. Para hacer software usamos un proceso que tiene un conjunto de etapas que generan salidas (artefactos) y son sw.

Clase 3:

El PUD/ RUP (proceso unificado de desarrollo) está formado por la etapa de requerimientos, análisis, diseño, implementación, prueba, despliegue -> cosas para hacer sw.

El sw es todo lo que se genera de las actividades que yo hago,

Disciplinas de gestión vinculadas al proyecto, las técnicas están vinculadas al producto. Y las de soporte (que incluye la gestión de configuración de sw, métricas (que se dividen en el dominio de procesos, proyectos o productos), ppqa)

El diseño de un prototipo es también sw.

Los cambios de un sw normalmente vienen del ambiente, también están nuestros errores o cosas que dejamos pendientes (desde el inicio podrían hacerse mejor, deudas técnicas)

Gestión de configuración; disciplina que intenta administrar los ítems de configuración a lo largo de su vida. Sirve para que el producto de sw tenga integridad. Para una configuración en particular, necesito;

Involucra para la configuración:

- ❖ Identificarla en un momento dado
- ❖ Controlar sistemáticamente sus cambios
- ❖ Mantener su integridad y origen

identificarla en un momento dado es la versión de todos los ítems.

Trazabilidad/ rastreabilidad: poder establecer vínculos de información y relacionamiento entre todo lo que conforma aquello que quiero seguir.

El primer artefacto que generamos es la ERS, hay una rastreabilidad pre ers (se usan para responder preguntas como quién te dijo que cambies algo, porque se hizo esto, etc (para saber el origen de algo) y post ers (todo lo que se genera a partir de la ers, modelo de análisis, de diseño, de arquitectura, de bd, código, casos de prueba, etc, arma vínculos para saber donde se analizó, donde se diseñó y como se probó)

La gestión de configuración nos permite volver a versiones anteriores. Vuelvo a la última línea base aceptada.

Para que el producto tenga integridad debe satisfacer requerimientos funcionales y no funcionales y ser rastreables.

Diferencia entre ítem y sw: el ítem si o si tiene que ser algo digital, el sw puede estar en papel, si algo está en papel le saco una foto y la guardo en el repositorio y transformo sw en un ítem.

Las herramientas de versionado hacen automáticamente el versionado. No tengo que ponerle por ejemplo en git versión 1.0, lo hace solo, si tengo que mostrárselo a alguien que no tiene acceso al repositorio ahí si le pongo en el nombre la versión.

La variante es un ítem de configuración aparte, y se le pone el mismo nombre de versión que la original solo que se especifica por ejemplo, variante de LINUX.

Yo me bajo la versión a mi espacio local y cuando termino le hago check in/commit.

El plan de gestión de configuración es el primer ítem que tengo que crear, y lo que tiene primero es la estructura, también tienen que estar los nombres de los involucrados y la conformación del control de cambios (su rol y su nombre), cuando se define que hay una nueva línea base.

El cuadro con todas las convenciones del repo va también en el plan de gestión de config

No es lo mismo un ítem que una línea base, ni tampoco es lo mismo una gestión de configuración que una línea base.

Línea base: ítem o conjunto de ítems que fueron validados y probados y conformados como punto de referencia para el resto del equipo. No necesariamente está formada por todos los ítems del repositorio ni por todas sus versiones. Todo el mundo que está afectado sobre el cambio de la línea base se le va a informar.

A lo largo del desarrollo del producto se puede tener más de una línea base.

La línea base también es un ítem de configuración. El control de cambios incluye a todos los involucrados con los cambios, necesitamos representantes del mismo equipo (analista funcional representante de los requerimientos, líder de la arquitectura-> líder técnico; el que hace de gestor de configuración, cliente (depende de la característica del cambio, si lo originó el cliente está bueno que este y para que sepa el impacto que va a tener el cambio). Nadie toca la línea base si alguien no autorizó el cambio.

El comité tiene que autorizar el cambio y ver si lo que se hizo fue lo que se autorizó.

No mezclar ítems de producto y de proyecto. El proyecto está formado por iteraciones (ciclo de vida en cascada tengo 1 iteración, incremental tengo más de 1). Cada iteración tiene un plan de iteración que su ciclo de vida es más corto que el del proyecto.

Ejemplo de un documento de producto ERS_clima.doc, el nombre del proyecto no sea el mismo que el del producto-> PP_matrix.doc (uno solo por proyecto), plan de iteración-> pi_matrix<<nro>>.doc

Una rama puede ser integrada o descartada.

Una variante de un ítem es un cambio que no requiere una nueva versión-> por ejemplo la versión 1,5 y 1,6 son dos variantes.

Caso de uso para una iteración es un ítem de producto e iteración. Si el ítem muere con la iteración entonces es de iteración, si muere con el proyecto es ítem de proyecto y si lo tengo que mantener mientras viva el producto es de producto.

Si el ítem se puede repetir se usan las etiquetas variables con <<>>

PRIMER TEMA DEL PARCIAL PRÁCTICO: USER STORIES

El agilismo adhiere a procesos empíricos

Pilares del empirismo: inspección, adaptación (necesidad de cambios), transparencia-> equipos empoderados y autogestionados.

Infografía: más imagen que texto

Valoramos más lo que está arriba que lo que está abajo.



Valoramos más individuos e interacciones que procesos y herramientas. Para nosotros es más importante la gente y los vínculos que se establecen entre ellos, el equipo debe poder resolver sus conflictos y autogestionarse.

Todas nuestras acciones deben apuntar a generar valor por ende, el software funcionando es más valorado que la documentación extensiva.

Valoramos más colaborar con el cliente (es parte del equipo, se trata del PO) por sobre negociaciones contractuales.

Scrum funciona a nivel de proyecto pero nunca dice que no haya que documentar

Principios más importantes:

- técnicos (diseñadores y desarrolladores entre otros) y no técnicos (responsables de negocios, sería en scrum el PO, maneja el presupuesto por ende es el que decide lo que se hace, decide el que no el cómo) trabajando juntos. La responsabilidad del producto la tiene el po y la responsabilidad de cómo se construye la tiene el equipo. Este principio es la clave del éxito.
- el mejor medio es la comunicación cara a cara
- Las mejores arquitecturas requisitos y diseños **emergen** de equipos autoorganizados. Las decisiones las toma el equipo y no un superior.

El agilismo es un pensamiento que define buenas prácticas para trabajar

Cuando hablamos de **requerimientos ágiles** nos focalizamos en el valor, resultados, porque tenemos que aceptar la realidad y empezar a fabricar el producto para ganar experiencia y obtener la retroalimentación. En cada iteración voy a trabajar ciertos requerimientos y solo de esos necesito el máximo detalle.

Principios ágiles que siguen los requerimientos ágiles: aceptar que los requisitos cambian, nuestra mayor prioridad es satisfacer al cliente, comunicación cara a cara (uno de los componentes fundamentales de la US es la conversación), etc.

El agilismo dice que hay que ser expertos en el negocio, debemos entenderlo mucho para generar un outcome. Y no necesitas tener el 100% del producto especificado para empezar a hacer el producto, no solo no hace falta sino que no es posible porque el 50% de los requerimientos emergen conforme el producto se esté construyendo.



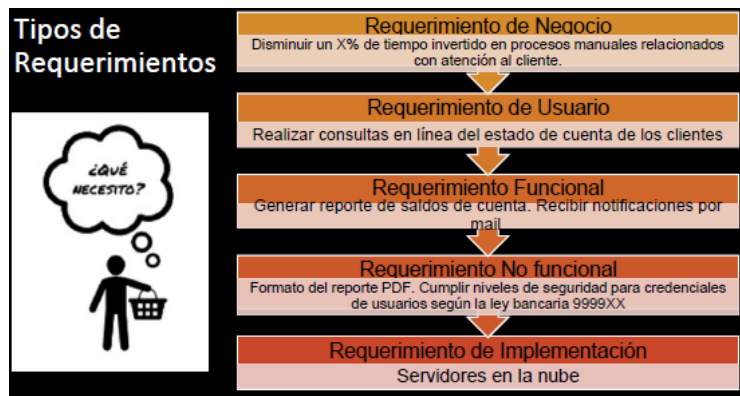
En PUD se definen los casos de uso que van en la iteración, y no se termina hasta no completarlos todos, en cambio, en scrum dejas fijo el tiempo y el alcance que voy a entregar tiene que ver con los requerimientos más arriba en el PB que voy a llegar a cumplir con el tiempo y recursos disponibles.

Artefacto principal del PO: product backlog, cada cosa del pb son ítem de product backlog. Cada uno elige la técnica que se usa para trabajar los ítem de pb (user stories, épicas, defectos, deudas técnicas). Las ideas menos formadas están más abajo.

El enfoque ágil es compatible con el lean por ende el agile toma lo de just in time que dice que no necesitamos conocer al principio todo el producto ni tenerlo especificado desde el principio

El product backlog nunca va a estar completo porque una vez que se implementó algo no vuelve a la lista, no es como una ERS.

Con los requerimientos ágiles apuntamos a atender: cuando trabajamos en ágil no profundizamos en los requerimientos del dominio de la solución. Usamos los requerimientos del negocio para a partir de ahí construir una solución que pueda entregársela al cliente con entregas tempranas.



User story: descripción corta de una funcionalidad que necesita el usuario, conformada por las 3 c: card, conversation (la más importante) y la confirmation (pruebas de aceptación, lo que tienes que identificar-> cómo lo vas a probar para demostrar que cumple con lo que el po necesita).

Está mal en la historia de usuario poner como usuario, necesito poner el rol de esa persona en el negocio.

La card es la parte visible de la historia de usuario, los criterios de aceptación tienen que ver con la confirmación.

Atrás van identificados los escenarios que el PO va a probar para confirmar si acepta o no el avance.

Una user es una funcionalidad que debe hacer algo de valor para el po por ende tiene que cortarse verticalmente, no puedo entregar la bd.

Un PO por producto, si no puede estar presente el po real se designa un representante que tenga conocimiento del negocio y pueda tomar decisiones, siempre tiene que venir del negocio.
Las pruebas de usuario no necesito que las escriba el usuario pero sí que las piense el usuario.

Las tech stories (los requerimientos no funcionales transversales, los que no son transversales se ponen como criterio de aceptación) no se entregan solas, ejemplo la performance o que sea mobile. Entonces cuando la voy a estimar voy a tener en cuenta todos los RNF que voy a necesitar.

Una US COMPLETA tiene la expresión, los criterios de aceptación, las pruebas (algunas que fallen y otras pasen) y la frase verbal

Las User Stories son Multipropósito

- Las historias son:
 - Una necesidad del usuario
 - Una descripción del producto
 - Un ítem de planificación
 - Token para una conversación
 - Mecanismo para diferir una conversación

Cuando la user esté en lo alto del PB se genera como un “token” para una conversación porque implica sentarse a hablar a refinar los requerimientos y también nos permite diferirla (ejemplo estoy armando el gps y me acuerdo de otra funcionalidad que más tarde definiremos mejor). Y sirve como ítem de planificación porque cuando vaya a implementar debo usarla para planificar.

La alta, baja y modificación son 3 user distintas.

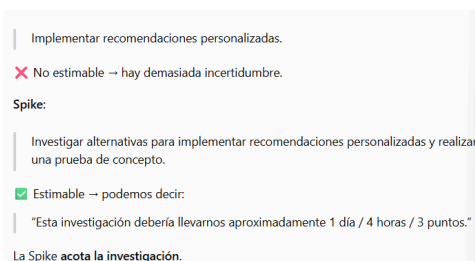
Tema: organizador de user, ejemplo tema clientes y ahí adentro van todas las users de los clientes

La definition of ready se define a nivel equipo y para ello se ayudan con invest.

Invest es para controlar lo mínimo que una user debe cumplir para estar en condiciones de desarrollarse.

SPIKE es una user que no se puede estimar, porque tiene problemas funcionales o de negocio que no están claras, o porque tiene problemas técnicos (combinación de tecnologías o cosas que no hiciste nunca o no se probaron nunca). Se ponen en el product backlog como spike y alguien debe investigarla y alguien resolverla (se puede hacer con un prototipo). La spike en sí debe ser estimable y debo poder

demostrar que resuelve el problema



El agilismo trabaja con gestión binaria asique o la user está hecha o nada, no gestiona porcentajes de avance para eliminar el “falta poco”.

Poner la user como usuario cuando son user de logueo por ejemplo pero sino no, ayuda a definir los roles y los permisos de cada rol

Una user story no es una especificación del requerimiento, para eso está el CU
No poner quiero poder como user, poner quiero o puedo

Los requerimientos son ágiles porque varían a lo largo de todo un proyecto.

Clase 1/9- estimaciones ágiles:

Estimación: se está trabajando sobre una incertidumbre, nos falta información (de la gente que va a trabajar, su nivel de expertis, los requerimientos, determinar efectivamente cual es el producto que el cliente necesita).

No es un compromiso, se confunde lo que quiero lograr con la estimación. Se tiene un factor de probabilidad, cualquier estimación debe ir acompañada de su probabilidad de cumplimiento.

Desperdicio: algo que no genera valor.

Ninguna decisión se puede tomar en un ambiente aislado del contexto-> no hay que ser dogmático, en función del contexto hacer las cosas.

El objetivo del negocio está formulado por alguien, por eso es subjetivo

Estimación es una predicción analítica o imparcial del esfuerzo o costo. La objetividad viene porque trato de usar parámetros lo más sistemáticos posibles para llegar a un número pero no se asocia a la certeza.

Compromiso: es lo que esperamos lograr con la planificación, la planificación toma como entrada una estimación, y la planificación incluye más aspectos que la estimación como: lineamientos del negocio. Cuanto más diferencia haya entre lo estimado y lo planificado, mayor riesgo. La planificación implica un compromiso, la estimación no.

Normalmente se parte de haber asociado estimación con compromiso, cronograma irreal dictado por el negocio. Cómo partiste de un cronograma subestimado (por ejemplo que el negocio te dijo que sea en menos tiempo, te fuerzan a hacer un cronograma ajustado a las necesidades del negocio), como consecuencia se recortan actividades tempranas por estar apurado-> viene el: lo hagamos así nomás total después lo mejoramos pero nunca hay tiempo para hacer eso entonces queda el código sucio y rápido y queda con defectos que se detectan en el testing o los detecta el cliente.

Se estima antes de arrancar un proyecto, sería como la pre estimación y después se estima en cada sprint para ver que se incorpore.

El cono de la incertidumbre: dice que las estimaciones se piden demasiado pronto porque el cliente las pide, se dan al principio para armar la propuesta al cliente y si después el cliente no acepta la propuesta se pierde la plata y el esfuerzo invertido, acá la incertidumbre es mayor. Agilismo propone que la certeza venga con la retroalimentación del software funcionando, entonces a más iteraciones - más certeza. muy pocos proyectos terminan consumiendo menos de lo que estimaron, y normalmente es porque arrancaron sobreestimados (decir que necesito más tiempo del que realmente necesito)

Buena estimación:

- expresada en rango, nunca decir voy a terminar el 8 de septiembre. Los rangos de oscilación tienen que ser razonables, un 20% para arriba y para abajo. (ejemplo entre 800 y 1200 horas)
- incluye un porcentaje de probabilidad
- la estimación no debe estar modificada para encajar en una fecha deseada

Porque son ágiles las estimaciones ágiles:

Porque estima el equipo de desarrollo entero y no una sola persona. Porque estimas a lo largo de todo el desarrollo iteración por iteración (en tradicional estimas 1 vez al principio, pueden existir revisiones y reestimaciones, pero la planificación y estimación inicial tienen mucho más peso y se intenta definir gran parte del proyecto anticipadamente). A medida que el equipo desarrolla y obtiene más información, puede cambiar sus estimaciones. También es porque estiman de forma relativa comparando US entre sí (tamaño, esfuerzo o complejidad), y no estiman poniéndoles un tiempo como en el modelo tradicional

Trabajan con estimaciones relativas: somos mejores estimando por comparación, el agilismo dice que no hagamos una estimación absoluta (cuantos metros de alto tiene el edificio, sin compararlo con otro).

La complejidad del software crece exponencialmente por eso usamos la escala de fibonacci (sumamos los dos anteriores para obtener el número)

La unidad de medidas para estimar las US son los story points (SP). Las us se estiman en sp. Nosotros estimamos el tamaño de la user, porque cuando estara o el precio puede derivar del tamaño.

Tamaño refiere al producto a construir y el esfuerzo se mide con horas, personas lineales (1 sola persona haciendo una sola cosa a la vez, no considera solapamientos). Esa estimación de esfuerzo se usa para estimar el tiempo (para responder el cuando va a estar). Primero se define el que, después el cómo y de ahí deriva el cuanto sale y cuanto tiempo lleva (en tradicional)

En ágil se dice que el que debe estimar es el que hace el trabajo (en tradicional estima el líder del proyecto y tiene la responsabilidad) y por eso son los equipos autogestionados.

Escala de estimación para estimar los ítems en el product backlog: talle de remera. Las dimensiones se usan para justificar porque sería un M o L.

Un punto de historia tiene 3 dimensiones que pueden presentarse en distintas proporciones (dimensiones) dependiendo de la user: complejidad, esfuerzo e incertidumbre.

La métrica de ágil es velocidad: cantidad de puntos de historia que el PO aceptó al final de la iteración (el acepta las us y eso suma sp) y esta no se estima, se calcula sumando y multiplicando al momento de presentar el incremento, osea al final de cada sprint.

Si tengo us1= 3 sp, us2= 5 sp, us3= 1sp y solo la us1 se aceptó entonces sumo 3, el resto vuelve al producto backlog.

Pero todavía no sabemos nuestra velocidad.

Hacemos el Sprint 1:

Completamos 20 puntos.

Sprint 2:

Completamos 18 puntos.

Sprint 3:

Completamos 22 puntos.

Sprint 4:

Completamos 20 puntos.

Nuestra velocidad está alrededor de:

20 puntos/Sprint

Ahora sí podemos hacer una predicción:

$100 \text{ puntos} \div 20 \text{ puntos/Sprint} = 5 \text{ Sprints}$

Si cada Sprint dura 2 semanas:

$5 \times 2 = 10 \text{ semanas aproximadamente.}$

Antes de tener velocidad real, podés hacer una previsión inicial usando una velocidad suposicional o esperada, pero va a ser menos confiable. Por eso Agile distingue:

Antes de empezar: "Creemos que aproximadamente serán 5 Sprints.

"Después de tener datos: "Según nuestra velocidad real, probablemente sean 5,2 Sprints."

Poker estimation: método que utiliza el equipo para asignar los Story Points a las User Stories.

El equipo elige una user canónica (se usa como referencia para comparar las otras user) y se le asocia un punto de historia. El ½ es para los equipos que usan 1 como canónica (la story más sencilla). Se estima en privado con el equipo para no sesgar ni verse sesgado por el resto. (el número que use para estimar debe estar en la escala de fibonacci, normalmente se usa fibonacci en las cartas pero puede ser que no) Cuando se llegó a un consenso se empieza el mismo proceso para la siguiente user, se corta cuando tengo la cantidad de historias suficientes para el sprint

Una user con 13 puntos normalmente no entraría en un sprint, hay que partirla

talle de remera para el product backlog y poker para el sprint backlog

Los Story Points son una medida relativa del tamaño de una User Story. No dicen: "Esta historia tarda 8 horas." Sino: "Esta historia es bastante más grande que una historia de 3 puntos." La secuencia de fibonacci se usa como escala para la story point, al igual que el tamaño de camisa. Son alternativas para expresar el tamaño relativo.

Reestimamos cuando como equipo notamos que se cometió un error, se re estima si cambio el tamaño de la user. Si llego al final del sprint, la user vuelve a ingresar al product backlog con el mismo tamaño que tenía

Se negocian los compromisos no las estimaciones.

La planificación del release debe responder cuantos sprints necesitas para entregar el mvp.
En scrum la duración del sprint es fija (máximo 1 mes), no se mueve. El agilismo dice que la duración es fija y el alcance es variable, si no terminaste todo entrega hasta donde tengas.

MVP: producto mínimo viable (factibilidad), para validar una hipótesis, alguien va a querer el producto, sirve para alguien?. El índice de retención va a determinar si es viable para determinar si vale la pena hacer el producto. Puede ser un prototipo, un video, una maqueta, solo quieres mostrarle a los clientes/ interesados para ver si el producto va a ser exitoso o no. El objetivo no es comercializarlo, solo se enfoca en el aprendizaje. Se define al comienzo del proyecto, cuando estás definiendo qué producto querés construir y cuál es el alcance inicial.

MMP: de comercializable, esto ya si tiene que ser software funcionando, es la cantidad mínima de características que el producto tiene que hacer para generar una versión que se pueda poner en producción, que tenga valor.

Todo el mundo habla de mvp cuando en realidad se refiere a mmp.

Hay que priorizar un conjunto limitado de funcionalidades.

Se lanza cuando definimos las condiciones para tener un mmp, ejemplo llegar a tal user.

MLP: la L es lovely, tiene que ver con que hay tanta oferta que hay una sola oportunidad para que el usuario disfrute usar el producto, no alcanza que tenga la funcionalidad sino que debe hacerlo feliz. Te lleva a tener en cuenta funcionalidades que antes decíamos que las veíamos después.

Defino aprox hasta que user es mi mvp y una vez que esta lo lanzas.

Puedo usar los 3 juntos o solo algunos.

Claves para desarrollar un mvp:

- que podamos obtener el feedback lo más rápido posibles
- seguir el ciclo construir, adaptar y cambiar-> viene por uno de los pilares del empirismo (sus pilares son: transparencia, adaptación)
- reducir los riesgos: busco con el menor costo, obtener la mayor info

Un mvp me permite visualizar lo que voy a desarrollar en un proyecto de software, y ver que tan grande es lo que voy a construir

En enfoque tradicionales se estima: tamaño (que tan grande va a ser lo que vamos a desarrollar, se mide en funcionalidades/ características), esfuerzo (horas personas lineales, no permite trabajo en paralelo, tiempo en que una persona tarda en hacer una actividad), costo, calendario - tiempo (días, meses)

En tradicional también se va estimando a lo largo de todo el proyecto.

En enfoque ágil se estima: tamaño de las user, la unidad de medida es en storys point.

Story point: es la suma de la complejidad (que tan difícil es el problema que trato de resolver, relacionada con la naturaleza del problema o con la tecnología necesitada), esfuerzo (hora lineales que me lleva desarrollar una user) e incertidumbre (nivel de conocimiento que tiene el equipo, desde el negocio hasta las tecnologías que necesito).

El número que se usa en poker planning puede seguir la escala lineal o de fibonacci o de talles de remera (acá usamos fibonacci) y se toma una referencia y en base a esa se compara el resto, en la cátedra se

toma como referencia la más fácil (con menor valor, que la combinación de esfuerzo, incertidumbre y complejidad sea baja).

Cuando estimamos ahí tenemos en cuenta la expertiz del equipo, no subestimar ni sobreestimar.

La clave de ponerle el valor es la justificación del porqué dio eso.

Cómo definir el mvp: debe estar declarada en forma de hipótesis, se usa la palabra alcance (las cosas que cubre y las que no). Lo vamos a definir por medio de un listado de frases verbales de las user stories. Se define; alcance: incluye (un listado, no incluye (módulo) cada uno con su justificación de porqué se incluye o no. Y si lo vuelvo a leer y me pregunto por qué?, la justificación está incompleta.

Clase 8/9 - componentes de proyecto:

El proyecto es una unidad de trabajo y es el medio por el cual se obtiene un producto.

Empírico: la actitud que tiene frente a los cambios es que rapido quiero retroalimentación para adaptar

Tipos de ciclo de vida: iterativo, secuencial, recursivo. Dentro del tipo de ciclo secuencial conocemos el cascada, dentro del iterativo está el iterativo incremental. **Revisar esto**

Diferencia entre proceso y ciclo de vida: El proceso define las cosas que hay que hacer (actividades, entradas, salidas, roles). Es la definición teórica. Mientras que el ciclo de vida del proyecto determina cuánto de cada actividad del proceso se hará cada vez, y los puntos de control (te dice como usar el proceso)

El PUD recomienda un ciclo de vida iterativo incremental.

En procesos empíricos, sólo se puede usar el ciclo iterativo porque se basa en generar conocimiento y experiencia rápidamente.

Si el resultado no es único, no es un proyecto. Además, debe tener una fecha de inicio y de fin.

Los proyectos deben tener un objetivo claro (todos entendemos lo mismo) y alcanzable (haciendo lo mejor que podemos todos, el objetivo puede lograrse). NO puede haber un objetivo alcanzable pero que no sea claro.

UN EJEMPLO ES ALGO CONCRETO, no hacerlos generales.

Lo importante de la planificación no es el plan de proyecto (artefacto), lo valioso es lo que aprendes mientras planificas.

En el paper de no silver bullet lo accidental apunta más a ayudar en el como y no en el que (el real problema)

Si agregamos personas a un proyecto atrasado, es peor porque gastamos tiempo en capacitarlos.

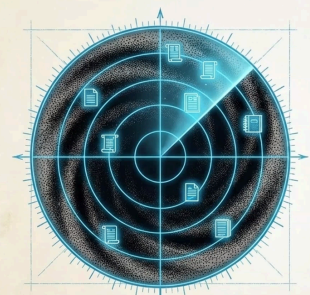
Hay una deformación personal al “y ya que estamos” entonces construimos desperdicio y perdemos el enfoque principal y sacrificamos velocidad en funciones rara vez usada.

Mientras más gente, más ruido de comunicación, más interpretaciones lo cual lleva a problemas.

LEER LA GUÍA DE SCRUM 2020 Y 2026, porque scrum da un minimo y maximo de gente?

La Hipótesis Documental: El papel como radar

En un mar de burocracia, un pequeño conjunto de documentos se convierte en los pivotes críticos del proyecto.



El Cuaderno de Trabajo (The Project Workbook)

- ☞ **El Acto de Escribir:** Requiere cientos de **mini-decisiones** que exponen políticas difusas.
- ☞ **El Mecanismo de Alerta:** Mantiene la vigilancia constante del estado del proyecto.
- ☞ **La Arquitectura de la Verdad:** El manual debe reflejar decisiones de diseño, fechas de revisión, y arquitectura formal vs. prosaica. Debe estar disponible para todos.

algo de información documentada tiene que

haber, el cómo no importa (sin en un pizarrón, foto o lo que sea), importa el que. La escritura requiere un nivel de precisión distinto que el habla.

Un producto de software tiene calidad si satisface las necesidades y expectativas del cliente

Disciplinas que integran la administración de proyectos: planificar y monitorear, se puede planificar y no monitorear pero no se puede monitorear sin planificar, porque el control es una actividad de comparación y necesito saber que tengo que comparar y un estándar contra qué compararlo.

Entonces, la administración se encarga de la planificación y monitoreo de un proyecto.

Las dimensiones sobre las que se mueve un proyecto son alcances (requerimientos), tiempo y costo (que son los recursos). La disciplina de gestión de proyectos busca un equilibrio entre las dimensiones, sin afectar la calidad. El cliente puede controlar el costo y el alcance, pero la única variable que debería controlar es la de que quiere (alcance)

En la gestión tradicional se derivan los recursos y el tiempo en función de los alcances. Y como el cliente puede venir y pedir algo más sin querer acceder a que se requiere más tiempo y recursos, se empieza a sacrificar la calidad.

No siempre que cambie algo voy a modificar las 3 dimensiones, puedo querer agregar algo y sacar otra cosa.

El líder de proyecto coordina y tiene a cargo el equipo del proyecto, habla con el cliente, con los jefes, etc (es la interfaz entre el equipo y los que tienen algo para decir).

En ágil se plantea que las personas sean multidisciplinarias, ejemplo que sean fullstack, que puedan negociar con el cliente, la documentación, etc. Dejar la user para cumplir con el criterio de ya terminado

Necesitamos un plan para que nos guíe el camino que vamos a emprender.

El alcance del proyecto y del producto, no son lo mismo. El alcance del proyecto se escribe en el plan del proyecto y el alcance del producto se especifica en la ERS. Se condicionan pero no son lo mismo.

ESTIMACIONES DE SOFTWARE

La gestión tradicional tiene más foco en la precisión de la estimación, apunta a estimaciones absolutas.

Debe seguir el siguiente orden

- Tamaño: del producto, apunta a saber qué se debe hacer, independiente de quién lo hace y la plata del cliente. En CU, requerimientos
- Esfuerzo: en base al tamaño determinó las horas persona lineales (1 sola persona trabaja haciendo una cosa a la vez)
- Calendario: transformo las horas en una estimación de cuándo, tengo en cuenta cantidad de personas, trabajo en paralelo, jornada de trabajo, disponibilidad del equipo. Derivo el calendario

- Costo: lo último a estimar porque se calcula en base a las anteriores estimaciones
- Recursos Críticos: no siempre están. Se calcula que se va a necesitar más de lo que se dispone

Se deriva el calendario a partir del esfuerzo

Riesgo tiene un factor de probabilidad y se mide por la exposición (que tiene en cuenta la probabilidad y el impacto, $\text{prob} \times \text{impacto}$). El riesgo se estima porque tiene una probabilidad. Riesgo no es problema, el hecho de que no alcance el tiempo es un problema, o empezar un proyecto que no se tiene en claro (no tiene probabilidad, hay certeza, es un problema). En un proyecto sano se espera una gestión proactiva de los riesgos (que también genera conocimiento). Algunos riesgos es que se vaya gente clave del proyecto, o tomar mal los requerimientos (el mas jodido), la tecnología
El cálculo de exposición sirve para comparar un riesgo con otro.